

我会把你的想法先压成一个可发表的“问题定义”，再展开成可预测的变量、系统闭环和几个具体课题方向。重点放在 retrieve/offloading 型 KV cache，而不是泛泛做 eviction。

已思考 5s >

你的直觉是对的：**retrieve 类 KV cache 的核心不应该只是“召回哪些 block”，而应该是“在每个解码步，动态决定召回预算、召回对象、召回时机、召回粒度和纠错策略”**。这比单纯 Top-K block retrieval 更像一个闭环控制问题。

我建议你研究思路定义成：

Query-/State-Aware Adaptive KV Retrieval：给定当前 query、生成状态、任务阶段和历史召回反馈，模型动态预测当前 token 需要访问多少 KV、访问哪些 KV、以什么粒度访问，以及是否需要纠错或回顾。

1. “预测每个 token 需要召回多少 block”是一个好切口

这可以叫 **dynamic retrieval budget prediction**。

现有很多方法默认固定 Top-K 页或固定 cache budget，例如每步召回 K 个 block。但实际上每个 token 对历史上下文的依赖强度不同：

- 标点、连接词、格式 token 可能只需要局部 KV；

- 答案中的实体 token 可能需要远程 evidence;
- 推理转折点可能需要重新访问早期 CoT;
- 总结任务中的句首 token 可能需要全局信息, 而句内 token 更多依赖局部上下文;
- 代码生成中函数调用、变量名、跨文件引用 token 的远程依赖强度不同。

所以可以预测:

$$K_t = f(q_t, h_t, \text{task state}, \text{retrieval history})$$

其中 K_t 是当前解码步需要召回的 block 数。

这个方向的卖点是: **现有 retrieve 方法大多动态选择“召回什么”, 但较少精细建模“召回多少”。**

2. 除了预测“召回多少 block”, 还可以预测什么?

我建议把可预测对象分成 6 类。

A. 预测 retrieval budget: 召回多少

这是你已经想到的。

可以预测:

- 当前 token 需要召回的 block 数 K_t
- 每一层需要召回的 block 数 $K_{t,l}$
- 每个 attention head 需要召回的 block 数 $K_{t,l,h}$
- 输入 KV 和输出 KV 的不同预算
- GPU resident cache 和 CPU recall cache 的配比

更细一点, 可以不是直接预测 K, 而是预测一个 **coverage target**:

当前 token 需要覆盖多少历史注意力质量?

例如预测需要覆盖 70%、90%、95% 的历史 attention mass, 然后再自适应选择 block 数。

这比固定 K 更自然, 因为不同 token 的注意力分布熵不同。注意力非常集中的 token 只需要少量 block; 注意力分散的 token 需要更多 block。

B. 预测 retrieval necessity: 这个 token 是否需要远程召回

不是每步都需要检索。

可以先做一个门控:

$$r_t \in \{0, 1\}$$

表示当前 token 是否触发远程 KV retrieval。

这对应 LouisKV / FreeKV 那类思路, 但可以做得更模型化: 不是靠语义边界或上一步 query 规则, 而是预测:

- 当前 token 是否只需要 sliding window;
- 是否需要访问远程 input context;
- 是否需要访问过去 generated tokens;
- 是否需要访问被 offload 到 CPU 的 KV;
- 是否应该跳过 retrieval, 沿用上一步召回结果。

这个方向很有系统价值，因为 retrieve 最大瓶颈之一是 **每步选择 + PCIe 传输**。如果你能减少触发频率，会直接影响端到端延迟。

可以命名为：

Token-wise Retrieval Trigger Prediction

C. 预测 retrieval granularity: 召回粒度

不一定每次都按固定 page/block 召回。

可以预测当前 token 适合哪种粒度：

- token-level
- block-level
- semantic chunk-level
- sentence-level
- CoT step-level
- variable-length temporal segment
- cluster-level

例如：

- factual QA: 更适合 sentence / evidence span;
- summarization: 更适合 semantic window;
- reasoning: 更适合 thought step;
- code: 更适合 symbol / function scope;
- dialogue: 更适合 utterance-level memory。

你的动态范式可以是：

$$g_t = f(q_t, h_t)$$

其中 g_t 是 retrieval granularity。

这会比单纯 page retrieval 更有新意，因为 Quest、ARKVALE、FreeKV 等很多方法还是强依赖 page/block 这一系统友好的粒度。你的贡献可以是 **系统友好粒度与语义友好粒度之间的动态切换**。

D. 预测 retrieval target type: 召回哪一类 KV

KV cache 不是同质的。可以把历史 KV 分成几类：

1. 输入上下文 KV
2. 输出生成 KV
3. 局部窗口 KV
4. 远程 evidence KV
5. reasoning / CoT KV
6. 格式控制 KV
7. 实体记忆 KV
8. 被压缩/合并/量化的 KV
9. 完整精度 KV

然后预测当前 token 主要需要哪一类。

例如对于 Long-CoT 模型：

- 深层推理 token: 召回 earlier reasoning states;

- 执行 token: 更多用局部 CoT;
- 答案 token: 召回 input evidence;
- 格式 token: 不用远程召回。

这可以形成一个更清楚的研究问题:

Can an LLM predict the functional role of the next decoding step and retrieve different KV sources accordingly?

这个比“预测多少 block”更高级, 也更适合写成方法框架。

E. 预测 retrieval timing: 什么时候提前召回

FreeKV 做了 speculative retrieval, 但你可以进一步泛化。

可以预测:

- 当前 token 是否马上需要 retrieval;
- 下一个 token 或未来若干 token 是否会需要 retrieval;
- 是否可以提前异步搬运;
- 上一步 retrieval 是否可以复用;
- 当前 query 是否处于语义边界, 需要刷新召回集合;
- 当前 query 是否仍在同一局部语义段内, 可以保持上一轮结果。

也就是说, 不只预测当前步的 Top-K, 而是预测一个 **retrieval horizon**:

$$H_t = \text{未来多少步可以复用当前召回集合}$$

如果 H_t 大, 你可以一次召回、多步复用, 降低传输与 selection 开销。

这可以成为一个很有系统味的创新点:

Horizon-aware KV retrieval: 模型预测召回结果的有效期。

F. 预测 error risk: 是否需要纠错 / 回顾

稀疏 retrieval 最大问题是近似误差积累。RetroAttention 试图回顾性修正历史 attention 输出。你可以做一个轻量版本:

预测当前 token 的 **retrieval risk**:

$$e_t = \Pr(\text{sparse retrieval will hurt output})$$

如果风险高, 则:

- 增大 retrieval budget;
- 使用更高精度 KV;
- 触发 full attention fallback;
- 召回更多输入 block;
- 使用 retrospective correction;
- 保留当前 token 的 query/output 以便未来修正。

这可以让系统从“固定稀疏”变成“风险自适应稀疏”。

3. 一个更完整的动态范式: 不要只做 Top-K Retrieval

我建议你方法设计成四个预测器:

1. Trigger Predictor: 是否检索

判断当前 step 是否需要远程 KV。

输出：

$$r_t \in \{0, 1\}$$

如果不检索，就只用 local window + resident cache。

2. Budget Predictor: 检索多少

输出当前 token 的 retrieval budget：

$$K_t$$

或者输出 attention mass coverage target：

$$\rho_t \in [0, 1]$$

然后根据 block score 累积到 ρ_t 为止，而不是固定 Top-K。

这个很关键。**不要只预测 K，最好预测 coverage target**，因为 K 受上下文长度、block size、任务类型影响太大。

3. Scope Predictor: 检索什么类型

输出当前需要的 KV scope：

$$s_t \in \{\text{input, output, CoT, local, entity, evidence}\}$$

例如：

- answer generation: 优先 input evidence;
- reasoning continuation: 优先 previous CoT;
- summarization: 优先 distributed semantic windows;
- code: 优先 definitions/imports/recent code。

4. Horizon Predictor: 检索结果复用多久

输出：

$$H_t$$

表示当前 retrieved KV set 可以复用多少步。

如果 $H_t > 1$ ，系统就不用每步重新 select + transfer。

这个点很重要，因为 retrieve 类方法的真正瓶颈往往不是 attention FLOPs，而是：

- block scoring;
- CPU-GPU transfer;
- synchronization;
- cache replacement;
- memory fragmentation。

4. 可以形成的核心方法：DynaKV-Retriever

你可以考虑一个框架名类似：

DynaKV-R: Dynamic Token-wise KV Retrieval for Long-Context LLM Inference

核心 pipeline：

1. 对所有 KV block 建立轻量 metadata
例如 min/max key boundary、centroid、norm、recency、semantic id、layer/head summary。
2. 当前解码步拿到 query state q_t 、hidden state h_t 、上一步 retrieval info。
3. 一个轻量 policy head 预测：
 - 是否检索 r_t
 - 检索预算 K_t 或 coverage ρ_t
 - 检索粒度 g_t
 - 检索范围 s_t
 - 复用 horizon H_t
 - 风险分数 e_t
4. 根据 metadata 对 candidate blocks 打分。
5. 按预测预算召回 block。
6. 做近似 attention。
7. 记录反馈信号，用于下一步 policy：
 - 实际 attention entropy；
 - top block concentration；
 - retrieved block hit rate；
 - local vs remote attention mass；
 - output confidence；
 - query drift；
 - semantic boundary change。

5. 训练信号怎么来？

这是决定论文强弱的地方。

你需要避免“我设计了几个 heuristic”这种感觉，最好有清晰的监督信号。

监督信号 1：来自 full attention oracle

离线跑 full attention，记录每个 token 的真实 attention 分布。

然后定义：

- 最小需要多少 block 才能覆盖 90% attention mass；
- 哪些 block 是 oracle important blocks；
- 当前 token 是否真的需要远程 block；
- 当前 token 的 attention entropy；
- 当前 retrieved set 可以被未来多少 token 复用。

标签可以是：

$$K_t^* = \min K \quad s.t. \quad \sum_{b \in \text{TopK blocks}} A_{t,b} \geq \rho$$

这里 $A_{t,b}$ 是当前 query 对 block b 的 attention mass。

这能直接训练 Budget Predictor。

监督信号 2: attention entropy

如果 attention 分布集中, K 小; 如果分散, K 大。

可以让模型预测 entropy:

$$\mathcal{H}_t = - \sum_b A_{t,b} \log A_{t,b}$$

然后把 entropy 映射成 budget。

这个比直接预测 K 更稳定。

监督信号 3: retrieval reuse horizon

定义未来若干步的 oracle top blocks 与当前 top blocks 的 Jaccard 相似度:

$$J(t, t + \Delta) = \frac{|B_t \cap B_{t+\Delta}|}{|B_t \cup B_{t+\Delta}|}$$

如果未来多步都相似, 说明当前 retrieval 可以复用。

这可以训练 Horizon Predictor。

监督信号 4: sparse-full discrepancy

比较 sparse attention output 和 full attention output 的误差:

$$\|o_t^{sparse} - o_t^{full}\|$$

误差高说明当前 token 是 high-risk token, 需要更多召回或纠错。

这可以训练 Risk Predictor。

6. 你可以主打的创新点

我建议不要把论文写成“又一个 KV retrieval top-k 方法”。更好的定位是:

创新点 1: 从 fixed-budget retrieval 到 token-adaptive retrieval

现有方法经常固定 K 或固定 cache ratio。你的方法让每个 token 自己决定 retrieval budget。

一句话表述:

Not every decoding step deserves the same KV budget.

创新点 2: 从 step-wise retrieval 到 horizon-aware retrieval

不是每个 token 都重新检索, 而是预测召回集合能复用多久。

一句话表述:

Retrieval results have temporal locality, and the model can predict their validity horizon.

创新点 3：从 object selection 到 policy prediction

传统方法重点是选哪些 block。你重点是学习一个 retrieval policy：

$$\pi_t = (r_t, K_t, g_t, s_t, H_t, e_t)$$

这更像一个动态控制器。

创新点 4：从 uniform KV 到 typed KV memory

不同历史 KV 有不同功能角色。你的系统可以区分 input evidence、generated reasoning、local syntax、format memory 等。

7. 一个可写成论文的具体题目

我认为最稳的方向是：

Adaptive Budgeted KV Retrieval for Long-Context LLM Inference

核心问题：

给定当前 decoding state，预测当前 token 所需的最小 KV retrieval budget，在保证输出质量的同时最小化 CPU-GPU transfer 和 attention compute。

方法包括：

- 1. block metadata index;
- 2. oracle attention mass label;
- 3. token-wise budget predictor;
- 4. trigger predictor;
- 5. horizon-based retrieval reuse;
- 6. risk-aware fallback。

这条线比较清楚，实验也好做。

8. 更强一点的版本：预测“召回多少 + 复用多久”

如果只做“预测每个 token 召回多少 block”，可能会被审稿人认为是 Quest/FreeKV 的增量改进。

我建议你加一个核心维度：

每次 retrieval 不只服务当前 token，而是服务未来一段 token。

也就是预测：

$$(K_t, H_t)$$

- K_t ：这次召回多少；
- H_t ：这次召回能复用多少步。

这样你的系统成本目标就变成：

$$\min \sum_t \text{AttentionCost}(K_t) + \text{TransferCost}(K_t, H_t)$$

其中 H_t 越大，单次 retrieval 的 amortized cost 越低。

这是 retrieve 类 KV cache 很关键但还没有被充分建模的点。

9. 推荐你的最终研究表述

你可以这样组织自己的研究 idea:

我想做一个动态 KV retrieval 范式。不同于现有方法在每个 decoding step 固定召回 Top-K KV blocks, 我认为当前 token 对历史 KV 的需求量、需求类型和需求持续时间都是动态变化的。因此, 我计划设计一个轻量级 retrieval policy predictor, 根据当前 query、生成状态、历史召回结果和 KV metadata, 同时预测: 是否需要召回、需要召回多少 block、应该召回哪类 KV、召回结果可以复用多久, 以及当前 sparse attention 是否存在高误差风险。该方法目标是在保持长上下文精度的同时, 降低不必要的 block scoring、CPU-GPU transfer 和 attention compute 开销。

这比“我想预测每个 token 需要召回多少 block”更完整。

10. 我最推荐你做的三个具体方向

方向一: Token-wise Dynamic Retrieval Budget

最直接, 易实现, 容易验证。

贡献点:

- 用 full attention oracle 生成每步最小 block budget 标签;
- 训练轻量 predictor;
- 动态调整 Top-K;
- 对比固定 K retrieval。

风险: 可能被认为增量, 需要强实验。

方向二: Horizon-aware KV Retrieval

更有系统创新。

贡献点:

- 预测当前 retrieved block set 可以复用多少步;
- 减少每步 selection 和 transfer;
- 适合 offloading/retrieval 场景;
- 可以和 FreeKV、LouisKV 形成直接对比。

风险: 训练 horizon 标签稍复杂, 但可做。

方向三: Risk-aware Retrieval with Adaptive Fallback

更稳精度。

贡献点:

- 预测 sparse attention 误差风险;
- 高风险 token 自动扩大 budget 或 fallback;
- 低风险 token 激进稀疏;
- 可以显著改善长生成误差累积问题。

风险: 需要设计好 risk label, 否则会变成 heuristic。

我的判断：**最有潜力的组合是“动态预算 + horizon 复用 + 风险门控”。**

不要一开始就做太复杂的语义类型、CoT 类型、任务类型全套预测。先把核心抽象压成：

$$\pi_t = (r_t, K_t, H_t, e_t)$$

也就是：

1. 要不要召回；
2. 召回多少；
3. 复用多久；
4. 风险多高。

这个足够形成一个清晰、动态、可实验验证的 KV retrieval 范式。

主要是关注block，不关注其他东西了；预测这个：当前 token 需要召回的 block 数；是否应该跳过 retrieval，沿用上一步召回结果；可以把历史 KV 分成几类，预测哪一类；

OK，怎么预测呢？有一个lightweight的模型吗？我希望这个lightweight的模型可以在模型推理时候自适应的更新，给我一个技术方案

我会把它收敛成一个只围绕 block 的方案：输入特征、轻量预测器、训练标签、推理时在线自适应更新，以及系统执行流程。重点回答“怎么预测”和“如何在线更新”。

已思考 13s >

可以。把问题收敛成 **block-level adaptive retrieval policy**，不碰 token-level、不碰复杂语义窗口。你的 lightweight model 只预测三个东西：

$$\pi_t = (K_t, r_t, c_t)$$

其中：

- K_t ：当前 token 需要召回多少个 block；
- r_t ：是否跳过 retrieval，沿用上一步 retrieved block set；
- c_t ：当前更需要哪一类历史 KV block。

我建议技术方案叫：

AdaBlock: Online-Adaptive Block Retrieval Policy for KV Cache

核心思想：

离线训练一个很小的 block policy predictor，推理时不更新 LLM 主模型，只在线更新这个 predictor 的校准参数 / gate / temperature，让它根据当前请求的注意力反馈自适应。

1. Block 分类：先把历史 KV block 分成几类

你不需要做复杂语义分类，先做系统友好的类别即可。

设历史 KV 被切成 block：

$$B = \{b_1, b_2, \dots, b_N\}$$

每个 block 有 metadata：

$$m_i = (\text{position}, \text{source}, \text{recency}, \text{attention_history}, \text{key_summary})$$

建议分成 5 类：

类别	含义	为什么需要
Local block	最近若干 block	保证局部连续性
Input block	prompt/input context 中的 block	QA、检索、证据定位需要
Generated block	已生成输出中的 block	长生成、代码、CoT 继续推理需要
Anchor block	特殊位置 block，如开头、system prompt、instruction、query block	常被模型反复引用
High-hit block	历史上被多次召回 / 高 attention 的 block	表示长期重要记忆

然后 lightweight model 不直接预测具体 block，而是先预测类别权重：

$$p_t^{cat} = [p_{local}, p_{input}, p_{gen}, p_{anchor}, p_{hit}]$$

再把总预算 K_t 分配给不同类别：

$$K_t^{(j)} = \lfloor K_t \cdot p_t^{cat}(j) \rfloor$$

最后每个类别内部根据 block score 选 Top- $K_t^{(j)}$ 。

这样好处是：**policy 只需要学习“当前 token 更依赖哪类历史”，具体 block 选择仍然由 Quest/ARKVALE 类 metadata scoring 完成。**

2. Lightweight model 的输入特征

不要把所有 block 都喂给小模型，否则开销太大。它应该只看 **query state + 全局统计特征 + 上一步反馈**。

当前步输入：

$$x_t = [q_t^{proj}, h_t^{proj}, \Delta q_t, s_t, a_{t-1}, u_{t-1}]$$

具体可以这样设计。

2.1 当前 query 特征

从当前层或若干层的 query 向量拿低维投影：

$$q_t^{proj} = W_q q_t$$

其中 $W_q \in \mathbb{R}^{d' \times d}$ ， d' 可以是 32 或 64。

为了省开销，建议只用：

- 最后一层 query；
- 或者每 4 层共享一个 query summary；
- 或者只用 pre-attention hidden state h_t 。

实际实现中：

$$z_t = \text{LayerNorm}(W_h h_t)$$

其中 z_t 是 64 维。

2.2 Query drift 特征：判断能否复用上一步 retrieval

这是预测“是否跳过 retrieval”的关键。

计算当前 query 与上一步 query 的变化：

$$d_t = 1 - \cos(q_t, q_{t-1})$$

也可以用低维投影：

$$d_t = 1 - \cos(z_t, z_{t-1})$$

如果 d_t 很小，说明当前 token 的注意力需求大概率没变，可以复用上一步 block set。

还可以加入 retrieved set 的稳定性：

$$J_t = \frac{|R_{t-1} \cap R_{t-2}|}{|R_{t-1} \cup R_{t-2}|}$$

如果前几步召回集合高度重叠，当前步也更可能跳过 retrieval。

2.3 Block score 统计特征：判断需要召回多少

你仍然用 metadata 给 block 打分：

$$s_{t,i} = \text{Score}(q_t, m_i)$$

例如 Quest 风格的 key min/max bound、centroid dot product、block key norm 等。

但是 lightweight model 不需要看所有 $s_{t,i}$ ，只看统计量：

$$\phi_t = [\max s, \text{mean top-8}, \text{mean top-32}, \text{score margin}, \text{entropy}, \text{tail mass estimate}]$$

最重要的是两个特征：

Top score margin

$$\text{margin}_t = s_{\text{top1}} - s_{\text{topK}}$$

如果 margin 大，说明少数 block 明显重要， K_t 可以小。

如果 margin 小，说明候选 block 接近， K_t 应该大。

Score entropy

$$H_t^{\text{score}} = - \sum_i \tilde{s}_{t,i} \log \tilde{s}_{t,i}$$

如果 entropy 高，说明注意力可能分散，需要更多 block。

2.4 历史反馈特征

推理过程中可以记录上一轮 sparse attention 的统计：

$$a_{t-1} = [\text{attention entropy}, \text{top block attention mass}, \text{local mass}, \text{remote mass}, \text{recalled block ut}]$$

例如：

$$\text{util}_{t-1} = \frac{\#\{b \in R_{t-1} : A_{t-1,b} > \epsilon\}}{|R_{t-1}|}$$

如果上一步召回了很多 block，但真正有 attention 的很少，说明 K_t 可以下降。

如果上一步 attention 很分散，说明 K_t 应该上升。

3. Lightweight model 架构

我建议不要用 Transformer，也不用复杂 MoE。用一个极小的 MLP 即可。

3.1 共享 encoder

$$e_t = \text{MLP}_{enc}(x_t)$$

结构：

```
Input dim: 128 ~ 256
Hidden dim: 128
Activation: SiLU / GELU
Output dim: 64
```

参数量可以控制在 50K 到 200K。

3.2 三个 prediction heads

Head 1: Reuse / Skip Predictor

输出是否复用上一步 retrieved block set:

$$p_t^{reuse} = \sigma(w_r^\top e_t)$$

如果：

$$p_t^{reuse} > \tau_r$$

则跳过 retrieval，直接用：

$$R_t = R_{t-1}$$

这一步可以省掉 block scoring、Top-K selection 和 CPU-GPU transfer。

Head 2: Budget Predictor

不要直接回归任意整数 K_t 。建议预测 **bucket**。

例如：

$$K_t \in \{0, 4, 8, 16, 32, 64, 128\}$$

输出：

$$p_t^K = \text{softmax}(W_K e_t)$$

然后取：

$$K_t = \arg \max_k p_t^K(k)$$

或者为了保守，用期望：

$$K_t = \sum_k p_t^K(k) K^{(k)}$$

实际系统里更推荐 bucket，因为 kernel 和 memory schedule 更容易优化。

Head 3: Block Category Predictor

输出 block 类别分布：

$$p_t^{cat} = \text{softmax}(W_c e_t)$$

例如：

$$p_t^{cat} = [0.2, 0.5, 0.1, 0.1, 0.1]$$

表示当前 token 主要需要 input blocks。

然后：

$$K_t^{input} = 0.5K_t$$

$$K_t^{local} = 0.2K_t$$

其余类别按比例分配。

4. 推理时完整流程

每个 decoding step 执行：

Step 1: 提取当前轻量特征

从 LLM 当前 hidden/query 中拿：

$$z_t = W_h h_t$$

计算 query drift：

$$d_t = 1 - \cos(z_t, z_{t-1})$$

读取上一轮 attention 反馈：

$$a_{t-1}$$

组成：

$$x_t = [z_t, d_t, \phi_t, a_{t-1}]$$

Step 2: 预测是否复用

$$p_t^{reuse} = \text{ReuseHead}(x_t)$$

如果复用：

$$R_t = R_{t-1}$$

并且跳过当前 block retrieval。

注意：复用不代表完全不变。可以保留 local block 滑动更新：

$$R_t = R_{t-1} \cup B_{local}$$

这样保证最近 token 总是可见。

Step 3: 如果不复用，预测预算和类别

$$K_t = \text{BudgetHead}(x_t)$$

$$p_t^{cat} = \text{CategoryHead}(x_t)$$

分配预算：

$$K_t^{(j)} = \lfloor K_t \cdot p_t^{cat}(j) \rfloor$$

Step 4: 类别内 Top-K block retrieval

对每个类别 j , 从该类别 block 中选：

$$R_t^{(j)} = \text{TopK}_{K_t^{(j)}} \{ \text{Score}(q_t, m_i) \}$$

最终召回：

$$R_t = \bigcup_j R_t^{(j)} \cup B_{local}$$

Step 5: 执行 sparse attention

只对：

$$R_t$$

中的 KV 做 attention。

Step 6: 记录反馈

记录：

- 当前 retrieved block 被实际 attention 使用的比例；
- attention entropy；
- top block attention mass；
- local vs remote attention mass；
- 当前 K_t 是否过大或过小的 proxy signal。

这些用于在线更新。

5. 离线训练标签怎么构造

最干净的方法是用 full attention oracle。

在训练数据上跑 full KV attention, 得到每个 token 对每个 block 的真实 attention mass：

$$A_{t,b}$$

其中：

$$A_{t,b} = \sum_{i \in b} A_{t,i}$$

5.1 Budget label

定义最小 block 数：

$$K_t^* = \min K \quad s.t. \quad \sum_{b \in \text{TopK}(A_{t,b})} A_{t,b} \geq \rho$$

例如 $\rho = 0.9$ 或 0.95 。

然后把 K_t^* 映射到 bucket:

$$K_t^* \rightarrow \{0, 4, 8, 16, 32, 64, 128\}$$

训练 Budget Head:

$$\mathcal{L}_K = \text{CE}(p_t^K, K_t^*)$$

5.2 Reuse label

判断当前 oracle important blocks 和上一步是否相似。

设:

$$O_t = \text{TopBlocks}_\rho(A_{t,b})$$

如果:

$$\frac{|O_t \cap O_{t-1}|}{|O_t \cup O_{t-1}|} > \eta$$

则:

$$r_t^* = 1$$

否则:

$$r_t^* = 0$$

训练:

$$\mathcal{L}_{reuse} = \text{BCE}(p_t^{reuse}, r_t^*)$$

5.3 Category label

每个 block 已经有类别 $c(b)$ 。

根据 oracle attention mass 计算每类的重要性:

$$y_t^{cat}(j) = \sum_{b:c(b)=j} A_{t,b}$$

归一化后作为 soft label:

$$\mathcal{L}_{cat} = \text{KL}(y_t^{cat} || p_t^{cat})$$

这个比 hard label 更好, 因为当前 token 可能同时需要 input block 和 generated block。

5.4 总 loss

$$\mathcal{L} = \lambda_1 \mathcal{L}_K + \lambda_2 \mathcal{L}_{reuse} + \lambda_3 \mathcal{L}_{cat}$$

可以加一个 cost-aware loss:

$$\mathcal{L}_{cost} = \alpha K_t - \beta \text{coverage}_t$$

让模型偏向更小预算。

6. 推理时如何自适应更新

你希望 lightweight model 在推理时自适应更新。这里要非常小心：**不能全量在线训练 MLP**，否则开销和稳定性都有问题。

我建议使用三层在线自适应，从安全到激进：

方案 A：只更新 calibration，不更新 MLP 主体

这是最稳的。

冻结 MLP 参数，只在线更新几个标量：

$$\theta_{online} = \{\tau_r, T_K, b_K, T_c, b_c\}$$

其中：

- τ_r : reuse 阈值;
- T_K : budget softmax temperature;
- b_K : budget bias;
- T_c : category softmax temperature;
- b_c : category bias。

例如 budget logits:

$$\ell_K = W_K e_t$$

在线校准后：

$$\ell'_K = \frac{\ell_K + b_K}{T_K}$$

如果系统发现当前请求中 sparse attention 风险较高，就增大 b_K ，让模型倾向于选更大的 K。

如果发现很多 block 没被使用，就减小 b_K 。

这是低风险、低开销、容易落地的方案。

在线反馈信号

推理时没有 full attention oracle，所以用 proxy。

反馈 1: retrieved block utilization

$$u_t = \frac{\#\{b \in R_t : A_{t,b} > \epsilon\}}{|R_t|}$$

如果 u_t 很低，说明召回太多：

$$b_K \leftarrow b_K - \eta$$

如果 u_t 很高且 attention 集中在边界 block，说明可能召回不够：

$$b_K \leftarrow b_K + \eta$$

反馈 2: attention tail pressure

看被召回 block 中排名靠后的 block 是否仍然获得较高 attention。

例如：

$$p_{tail} = \sum_{b \in \text{bottom 25\% of } R_t} A_{t,b}$$

如果 p_{tail} 高，说明当前 retrieval 截断可能太激进，需要增大 K。

反馈 3: score boundary margin

看最后一个被召回 block 和第一个未召回 block 的分数差：

$$\Delta_t = s_K - s_{K+1}$$

如果 Δ_t 很小，说明边界不确定，需要增大 K。

反馈 4: reuse failure signal

如果复用上一步 block set 后，attention 分布高度异常，例如：

- local block attention 突然升高；
- top attention mass 过低；
- entropy 过高；
- output confidence 下降；

则降低 reuse 概率：

$$\tau_r \leftarrow \tau_r + \eta$$

或者给 reuse head 加负 bias：

$$b_r \leftarrow b_r - \eta$$

方案 B：在线更新一个很小的 adapter

如果你想更像“模型推理时自适应更新”，可以加一个 tiny adapter，但只更新 adapter。

原始 encoder：

$$e_t = \text{MLP}_{\text{frozen}}(x_t)$$

加入在线 adapter：

$$e'_t = e_t + A_{\text{online}} \sigma(B_{\text{online}} e_t)$$

其中 bottleneck 很小，例如 rank = 4 或 8。

只更新：

$$A_{\text{online}}, B_{\text{online}}$$

但我建议再加约束：

$$\|A_{\text{online}} B_{\text{online}}\|_F < \epsilon$$

防止在线漂移太大。

在线 loss 不用 full attention, 用 proxy loss:

$$\mathcal{L}_{online} = \alpha \cdot \text{OverRetrieve} + \beta \cdot \text{UnderRetrieve} + \gamma \cdot \text{ReuseFailure}$$

其中:

$$\begin{aligned}\text{OverRetrieve} &= \max(0, u_{target}^{low} - u_t) \\ \text{UnderRetrieve} &= \max(0, p_{tail} - p_{tail}^{target}) \\ \text{ReuseFailure} &= 1[r_t = 1] \cdot H(A_t)\end{aligned}$$

这个方案有更强的自适应性, 但审稿时需要证明在线更新稳定、开销小。

方案 C: Contextual Bandit 在线更新

这是最有“动态决策”味道的方案。

把每个 action 定义为:

$$a_t = (r_t, K_t, c_t)$$

例如:

```
reuse or retrieve
K in {4, 8, 16, 32, 64}
category mixture in {local-heavy, input-heavy, gen-heavy, balanced}
```

reward 定义为:

$$R_t = -\lambda_1 \cdot \text{latency}_t - \lambda_2 \cdot K_t - \lambda_3 \cdot \text{transfer}_t - \lambda_4 \cdot \text{risk}_t$$

其中 risk 用 proxy:

$$\text{risk}_t = \text{tail pressure} + \text{high entropy} + \text{low confidence}$$

然后用 LinUCB / Thompson Sampling 更新一个轻量 policy。

优点:

- 很符合“推理时自适应”;
- 不需要 full attention label;
- 可以按请求动态学习。

缺点:

- 实验复杂;
- reward 设计容易被质疑;
- 需要避免 exploration 伤害生成质量。

我建议作为增强模块, 不作为主方法。

7. 我最推荐的最终设计

不要一开始就做 B 或 C。最稳的论文方案是:

Offline-trained lightweight policy + online calibration update

也就是:

1. 离线用 full attention oracle 训练 MLP;
2. 推理时冻结 MLP;

3. 在线只更新几个 calibration 参数;
4. 用 attention utilization、tail pressure、reuse failure 来调节预算和复用阈值。

这既能声称 **online-adaptive**, 又不会引入过高风险。

8. 具体算法

可以写成这样。

AdaBlock Policy

输入:

- 当前 hidden state h_t
- 当前 query projection z_t
- block metadata scores $s_{t,i}$
- 上一步 retrieved set R_{t-1}
- 上一步 attention feedback f_{t-1}

输出:

- reuse probability p_t^{reuse}
- budget distribution p_t^K
- category distribution p_t^{cat}

执行:

1. Extract lightweight state x_t .
2. Predict p_{reuse} , p_K , p_{cat} .
3. If $p_{reuse} > \tau_r$:
 $R_t = R_{t-1} \cup \text{LocalBlocks}$
- Else:
 $K_t = \text{BudgetBucket}(p_K)$
 Allocate K_t across block categories using p_{cat} .
 Retrieve top blocks within each category.
4. Run sparse attention on R_t .
5. Collect feedback: utilization, entropy, tail pressure, boundary margin.
6. Update online calibration parameters.

9. 在线校准规则可以非常简单

例如维护一个 budget bias:

$$b_K$$

最终预算:

$$K_t = \text{Bucket}(\text{BudgetHead}(x_t) + b_K)$$

更新规则:

```
if tail_pressure > high_threshold:
    b_K += eta
elif utilization < low_threshold:
    b_K -= eta
```

reuse 阈值:

$$\tau_r$$

更新规则:

```
if reused and attention_entropy is high:
    tau_r += eta
elif retrieved sets are stable for several steps:
    tau_r -= eta
```

类别 bias:

$$b_c$$

如果当前 attention mass 主要落在某类 block, 比如 input block:

$$b_c[input] += \eta$$

如果某类 block 被召回但几乎没被用:

$$b_c[j] - = \eta$$

这样模型会在一个 request 内快速适应:

- 当前任务是 QA;
- 当前任务是 summarization;
- 当前任务是 long-CoT;
- 当前任务是代码生成;
- 当前任务主要依赖 input 还是 generated history。

10. 为什么这个方案合理

你的 lightweight model 不直接解决所有问题, 而是拆成两层:

第一层: Block scoring

负责回答:

哪些 block 可能重要?

这个可以沿用 Quest / ARKVALE / FreeKV 那类 metadata scoring。

第二层: Policy prediction

负责回答:

当前 token 应该召回多少? 要不要复用? 该偏向哪类 block?

你的贡献在第二层。

这会让论文定位很清楚:

Existing KV retrieval methods focus on block ranking, while we focus on adaptive retrieval policy learning.

11. 实验设计

至少需要四组 ablation。

11.1 固定 K vs 动态 K

对比:

- fixed K = 16
- fixed K = 32
- fixed K = 64

- dynamic K

指标:

- accuracy / perplexity / task score
- average retrieved blocks
- latency
- PCIe transfer volume
- attention FLOPs

11.2 不复用 vs reuse predictor

对比:

- every-step retrieval
- heuristic reuse by query cosine
- learned reuse predictor
- learned reuse + online calibration

重点看:

- retrieval frequency 降低多少;
- latency 降低多少;
- 精度是否稳定。

11.3 无类别 vs 类别预测

对比:

- global Top-K
- category-balanced Top-K
- predicted category allocation

验证你的 c_t 是否有用。

11.4 无在线更新 vs 在线更新

对比:

- frozen policy
- online threshold calibration
- online adapter
- bandit variant

证明在线自适应确实提升不同任务/不同长度/不同模型下的鲁棒性。

12. 最小可行版本

如果你现在要开工, 我建议 MVP 这样做:

MVP 预测目标

只预测:

$$(K_t, r_t, p_t^{cat})$$

MVP 模型

```
Input:
- projected hidden state: 64 dim
- query drift: 1 dim
- top score stats: 16 dim
- previous attention feedback: 16 dim
- previous action: 8 dim
```

```
MLP:
- Linear 105 -> 128
- GELU
- Linear 128 -> 64
- GELU
```

```
Heads:
- reuse head: Linear 64 -> 1
- budget head: Linear 64 -> 7
- category head: Linear 64 -> 5
```

总参数不到 30K 到 50K。

MVP 在线更新

只更新：

```
budget_bias
reuse_threshold
category_bias
```

不要先做 adapter。

13. 这个方案的核心卖点

可以这样总结成论文贡献：

1. Token-adaptive block budget prediction

每个 decoding token 动态决定需要召回多少 block，而不是使用固定 Top-K。

2. Retrieval reuse prediction

学习判断当前 token 是否可以沿用上一步 retrieved blocks，从而减少每步检索和传输。

3. Category-aware block allocation

将历史 KV block 分成 local/input/generated/anchor/high-hit 等类别，并预测当前 token 对不同类别的需求。

4. Online calibration for request-level adaptation

在推理过程中利用 sparse attention 反馈在线调整 budget、reuse 和类别偏置，使策略适应不同任务和上下文分布。

14. 我建议你最终做的版本

最推荐的技术路线是：

Offline Oracle Training + Lightweight Policy MLP + Online Calibration + Category-aware Block

不要一上来就做复杂的 RL 或在线 adapter。先把下面三件事做扎实：

1. 动态 K 比固定 K 更省且精度不掉；
2. reuse predictor 能显著减少 retrieval 次数；
3. online calibration 能跨任务稳定。

这条路线足够形成一个完整的 retrieve 类 KV cache 研究方案。

训练的时候拿什么做监督信息：attention weight mass，
这个应该就可以
但是test time能用什么信息更新呢？
训练和测试的时候分别用什么loss？

已思考若干秒 >

对。训练时用 full-attention 的 block attention mass 做监督是最干净的。测试时不能拿 full attention，所以只能用 sparse attention 内部可观测的 proxy signal 做在线更新。

你这个方法的关键要分清楚：

阶段	能看到什么	不能看到什么	目标
训练	full attention mass $A_{t,b}^{full}$	无	学到 oracle retrieval policy
测试	sparse attention 内部统计、block score、生成置信度、历史行为	full attention mass	在线校准 policy，避免过取/欠取

1. 训练时监督信息：block attention mass

假设一个 token t ，历史 KV 被切成 block：

$$\mathcal{B} = \{b_1, b_2, \dots, b_N\}$$

full attention 下，token t 对每个历史 token i 有 attention weight：

$$a_{t,i}$$

把 token-level attention 聚合成 block-level mass：

$$A_{t,b} = \sum_{i \in b} a_{t,i}$$

如果多头多层，可以先做聚合：

$$A_{t,b} = \frac{1}{LH} \sum_{l=1}^L \sum_{h=1}^H \sum_{i \in b} a_{t,i}^{l,h}$$

但我建议一开始不要对所有层所有头做。更实际的是：

- 选最后几层；
- 或者选 retrieval 发生的关键层；
- 或者按 layer group 聚合。

否则监督信号会很噪。

2. 训练时分别构造三个标签

你现在要预测三个东西：

1. 当前 token 需要召回多少 block；
2. 是否应该跳过 retrieval，沿用上一步召回结果；

3. 当前 token 主要需要哪一类 block。

对应三个标签。

2.1 Budget label: 需要召回多少 block

定义 oracle important block set。

先按 $A_{t,b}$ 从大到小排序：

$$b_{(1)}, b_{(2)}, \dots, b_{(N)}$$

然后找最小的 K_t^* ，使得 Top- K block 覆盖足够多的 attention mass：

$$K_t^* = \min K \quad \text{s.t.} \quad \sum_{j=1}^K A_{t,b_{(j)}} \geq \rho$$

其中 ρ 可以是：

- 0.8：更激进，更省；
- 0.9：比较均衡；
- 0.95：更保守，更准。

然后把 K_t^* 映射到 bucket：

$$K_t^* \in \{0, 4, 8, 16, 32, 64, 128\}$$

训练 budget head：

$$\mathcal{L}_K = \text{CE}(p_t^K, y_t^K)$$

其中 y_t^K 是 bucket label。

2.2 Reuse label: 是否可以沿用上一步 retrieval

你的真实问题是：

当前 token 的重要 block 集合和上一步是否足够相似？

定义当前 token 的 oracle block set：

$$O_t = \{b_{(1)}, \dots, b_{(K_t^*)}\}$$

上一步：

$$O_{t-1}$$

计算 Jaccard 相似度：

$$J_t = \frac{|O_t \cap O_{t-1}|}{|O_t \cup O_{t-1}|}$$

如果：

$$J_t > \eta$$

则认为可以 reuse：

$$y_t^{reuse} = 1$$

否则：

$$y_t^{reuse} = 0$$

例如：

- $\eta = 0.7$: 严格;
- $\eta = 0.5$: 宽松。

训练 reuse head:

$$\mathcal{L}_{reuse} = \text{BCE}(p_t^{reuse}, y_t^{reuse})$$

不过这里有一个细节：**reuse label 不应该只看 set overlap, 也要看 coverage loss。**

更合理的定义是：

$$C_t^{reuse} = \sum_{b \in O_{t-1}} A_{t,b}$$

表示如果当前 token 沿用上一步 block set, 可以覆盖多少当前 full attention mass。

如果：

$$C_t^{reuse} \geq \rho_{reuse}$$

则：

$$y_t^{reuse} = 1$$

否则：

$$y_t^{reuse} = 0$$

这个比 Jaccard 更直接，因为你关心的是 attention coverage，而不是集合形式。

我更推荐这个定义。

2.3 Category label: 当前 token 需要哪类 block

假设 block 类别为：

$$c(b) \in \{\text{local}, \text{input}, \text{generated}, \text{anchor}, \text{high-hit}\}$$

用 full attention mass 聚合到类别：

$$y_t^{cat}(j) = \sum_{b:c(b)=j} A_{t,b}$$

然后归一化：

$$\tilde{y}_t^{cat}(j) = \frac{y_t^{cat}(j)}{\sum_{j'} y_t^{cat}(j')}$$

这是一个 soft label。

训练 category head:

$$\mathcal{L}_{cat} = \text{KL}(\tilde{y}_t^{cat} \| p_t^{cat})$$

不要用 hard label，因为一个 token 往往同时依赖 local 和 input blocks。

3. 训练时总 loss

基础版本：

$$\mathcal{L}_{train} = \lambda_K \mathcal{L}_K + \lambda_R \mathcal{L}_{reuse} + \lambda_C \mathcal{L}_{cat}$$

更好一点，加入 cost-aware regularization：

$$\mathcal{L}_{train} = \lambda_K \mathcal{L}_K + \lambda_R \mathcal{L}_{reuse} + \lambda_C \mathcal{L}_{cat} + \lambda_{cost} \mathbb{E}[K_t]$$

其中 $\mathbb{E}[K_t]$ 是模型预测的期望召回 block 数：

$$\mathbb{E}[K_t] = \sum_m p_t^K(m) K_m$$

这个项会鼓励模型不要过度召回。

但注意： λ_{cost} 不能太大，否则会欠取，精度掉。

4. 测试时能用什么信息更新？

测试时看不到 full attention mass，但可以看到以下信息：

4.1 Sparse attention mass

虽然没有 full attention，但你可以看到在当前 retrieved blocks 内部的 attention：

$$\hat{A}_{t,b}, \quad b \in R_t$$

也就是 sparse attention 后，对已召回 block 的注意力分布。

这能提供几个 proxy。

4.2 Utilization：召回 block 是否真的被用上

定义：

$$u_t = \frac{|\{b \in R_t : \hat{A}_{t,b} > \epsilon\}|}{|R_t|}$$

如果 u_t 很低，说明召回太多。

例如召回了 64 个 block，只有 8 个 block 有明显 attention，那 K_t 可能过大。

4.3 Tail pressure：边缘 block 是否仍然重要

将召回 block 按 score 排序，取最后 25% 的 block：

$$R_t^{tail}$$

计算：

$$P_t^{tail} = \sum_{b \in R_t^{tail}} \hat{A}_{t,b}$$

如果 tail block 仍然有较高 attention mass，说明当前预算可能不够，因为更后面的未召回 block 可能也重要。

这是测试时判断 **under-retrieval** 的重要 proxy。

解释：

- P_t^{tail} 低：说明召回集合边缘已经不重要，K 够了甚至太大；
- P_t^{tail} 高：说明召回截断处仍然有注意力，K 可能太小。

4.4 Boundary margin: 第 K 个和第 K+1 个 block score 差距

虽然未召回 block 没有 attention, 但你通常有 metadata score:

$$s_{t,b}$$

如果当前选择 Top- K , 可以看:

$$\Delta_t = s_{t,(K)} - s_{t,(K+1)}$$

如果 Δ_t 很小, 说明边界不稳定, Top-K 和 Top-(K+1) 没明显区别。此时 K 应该适当增大。

如果 Δ_t 很大, 说明重要 block 已经和不重要 block 拉开差距, K 可以减小。

4.5 Sparse attention entropy

计算召回 block 内部 attention entropy:

$$H_t^{sparse} = - \sum_{b \in R_t} \hat{A}_{t,b} \log \hat{A}_{t,b}$$

如果 entropy 很高, 说明注意力分散, 可能需要更多 block。

如果 entropy 很低, 说明少数 block 主导, 预算可以小。

4.6 Reuse failure signal

如果当前步选择 reuse:

$$R_t = R_{t-1}$$

但出现以下现象:

- sparse attention entropy 高;
- attention 大量集中在 local block;
- tail pressure 高;
- logits confidence 下降;
- 当前 query drift 大;

则说明 reuse 可能失败。

可以定义:

$$F_t^{reuse} = \alpha H_t^{sparse} + \beta P_t^{tail} + \gamma d_t - \delta \text{confidence}_t$$

如果 F_t^{reuse} 高, 则下一步提高 retrieval 触发概率, 降低 reuse。

4.7 生成置信度

可以用 next-token distribution 的 entropy:

$$H_t^{vocab} = - \sum_v p_t(v) \log p_t(v)$$

或者 top-1 / top-2 margin:

$$M_t^{vocab} = p_t(v_1) - p_t(v_2)$$

如果模型对 next token 很不确定，且 sparse attention 也分散，可以增大 retrieval budget。

但是这个信号比较弱，因为语言不确定性不一定来自 KV 缺失，也可能是任务本身多解。

所以它只能作为辅助项。

5. 测试时更新什么？

我建议测试时不要直接更新整个 MLP。更新三类轻量参数：

$$\theta_{online} = \{b_K, b_R, b_C\}$$

其中：

- b_K : budget bias;
- b_R : reuse bias / threshold;
- b_C : category bias.

5.1 Budget 在线更新

模型原始输出：

$$p_t^K = \text{softmax}(\ell_t^K)$$

在线校准后：

$$p_t^K = \text{softmax}(\ell_t^K + b_K)$$

如果 b_K 是标量，可以整体偏向更大的 K bucket。

更精细地，可以让 b_K 是每个 bucket 一个 bias：

$$b_K \in \mathbb{R}^{|\mathcal{K}|}$$

当欠取时，增加大 K bucket 的 bias；当过取时，增加小 K bucket 的 bias。

Budget test-time loss

定义测试时的 proxy loss：

$$\mathcal{L}_K^{test} = \alpha \cdot \mathcal{L}_{under} + \beta \cdot \mathcal{L}_{over}$$

其中：

$$\mathcal{L}_{under} = \max(0, P_t^{tail} - \tau_{tail}) + \max(0, H_t^{sparse} - \tau_H) + \max(0, \tau_\Delta - \Delta_t)$$

表示欠取风险。

$$\mathcal{L}_{over} = \max(0, \tau_u - u_t) + \lambda_K \frac{K_t}{K_{max}}$$

表示过取浪费。

整体：

$$\mathcal{L}_K^{test} = \alpha [\max(0, P_t^{tail} - \tau_{tail}) + \max(0, H_t^{sparse} - \tau_H) + \max(0, \tau_\Delta - \Delta_t)] + \beta [\max(0, \tau_u - u_t) +$$

直观地：

- tail pressure 高 $\rightarrow$ K 增大;
- entropy 高 $\rightarrow$ K 增大;
- boundary margin 小 $\rightarrow$ K 增大;
- utilization 低 $\rightarrow$ K 减小;
- K 本身太大 $\rightarrow$ K 减小。

5.2 Reuse 在线更新

reuse head 原始输出:

$$p_t^{reuse} = \sigma(\ell_t^R)$$

在线校准后:

$$p_t^{reuse} = \sigma(\ell_t^R + b_R)$$

如果 reuse 失败, 则降低 b_R , 让模型更少复用。

如果连续多步 retrieval set 稳定, 则提高 b_R , 让模型更多复用。

Reuse test-time loss

$$\mathcal{L}_R^{test} = 1[reuse] \cdot [\alpha H_t^{sparse} + \beta P_t^{tail} + \gamma d_t + \delta H_t^{vocab}] + 1[retrieve] \cdot [\eta \cdot 1(\text{retrieval set stable})]$$

更直白地:

- 如果 reuse 后 attention 分散、tail pressure 高、query drift 大 $\rightarrow$ 惩罚 reuse;
- 如果重新 retrieval 后发现 R_t 和 R_{t-1} 高度重合 $\rightarrow$ 说明其实可以 reuse, 鼓励 reuse。

retrieval set 稳定性:

$$S_t = \frac{|R_t \cap R_{t-1}|}{|R_t \cup R_{t-1}|}$$

如果:

$$S_t > \tau_S$$

则说明可以提高 reuse 概率。

5.3 Category 在线更新

category head 原始输出:

$$p_t^{cat} = \text{softmax}(\ell_t^C)$$

在线校准后:

$$p_t^{cat} = \text{softmax}(\ell_t^C + b_C)$$

测试时可以观察: 被召回的每类 block 实际获得了多少 sparse attention mass。

$$\hat{A}_t^{cat}(j) = \sum_{b \in R_t, c(b)=j} \hat{A}_{t,b}$$

这虽然不是 full attention category mass, 但可以作为局部反馈。

Category test-time loss

$$\mathcal{L}_C^{test} = \text{KL}(\hat{A}_t^{cat} \| p_t^{cat})$$

也就是说，如果当前召回后发现 input block 获得了大量 attention，而模型原来给 input 类别分配很少，那么下一步提高 input bias。

但是要注意：这个信号有选择偏差。因为没召回的类别没有 attention，所以不能完全相信。

解决方法：

- 只做慢速 EMA 更新；
- 不对未召回类别强惩罚；
- 对类别 bias 设置上下界。

更新：

$$b_C \leftarrow (1 - \mu)b_C + \mu(\hat{A}_t^{cat} - p_t^{cat})$$

6. 测试时总 loss

可以写成：

$$\mathcal{L}_{test} = \lambda_K \mathcal{L}_K^{test} + \lambda_R \mathcal{L}_R^{test} + \lambda_C \mathcal{L}_C^{test}$$

但是实现上不一定真的要 backprop。你可以用规则更新，这样更快更稳定。

7. 推荐的在线更新方式：不要反传，直接规则校准

我建议你论文主方法用 **gradient-free online calibration**，因为系统开销低。

维护：

$$b_K, b_R, b_C$$

每步更新：

Budget bias

```
if tail_pressure > tau_tail or boundary_margin < tau_margin:
    increase large-K bias
elif utilization < tau_util:
    increase small-K bias
```

数学形式：

$$b_K[large] \leftarrow b_K[large] + \eta \cdot 1[P_t^{tail} > \tau_{tail}]$$
$$b_K[small] \leftarrow b_K[small] + \eta \cdot 1[u_t < \tau_u]$$

Reuse bias

```
if reused and reuse_failure high:
    b_R -= eta
elif retrieved and retrieval_set_overlap high:
    b_R += eta
```

数学形式：

$$b_R \leftarrow b_R - \eta F_t^{reuse}$$
$$b_R \leftarrow b_R + \eta 1[S_t > \tau_S]$$

Category bias

```
category_attention = sparse attention mass per recalled category
b_C = EMA(b_C + eta * (category_attention - predicted_category))
```

即：

$$b_C \leftarrow \text{clip} \left(b_C + \eta (\hat{A}_t^{cat} - p_t^{cat}) \right)$$

8. 训练 loss 和测试 loss 的对应关系

可以这样写论文逻辑：

预测目标	训练监督	训练 loss	测试反馈	测试更新
K_t	full attention coverage 最小 K	CE / ordinal loss	tail pressure、utilization、boundary margin、sparse entropy	调整 budget bias
reuse	上一步 block set 对当前 full attention 的 coverage	BCE	query drift、reuse 后 entropy、retrieval set overlap	调整 reuse bias
category	full attention mass 按类别聚合	KL	recalled category 的 sparse attention mass	调整 category bias

9. 一个更好的 Budget loss: ordinal loss

因为 K bucket 有顺序：

$$4 < 8 < 16 < 32 < 64$$

普通 CE 不知道 16 和 32 比 16 和 128 更接近。

所以 Budget Head 更适合用 ordinal regression。

做法：预测多个二分类：

$$P(K_t > 4), P(K_t > 8), P(K_t > 16), \dots$$

训练 loss：

$$\mathcal{L}_K^{ord} = \sum_m \text{BCE}(P(K_t > K_m), 1[K_t^* > K_m])$$

这样模型更稳定。

我建议最终论文里用 ordinal budget loss，而不是普通 CE。

10. 还可以加一个 coverage-aware training loss

如果你训练时直接让模型输出 budget，然后根据 predicted budget 选择 block，可以计算 predicted set 的 oracle coverage：

$$C_t(\hat{R}_t) = \sum_{b \in \hat{R}_t} A_{t,b}^{full}$$

希望 coverage 高，同时 K 小：

$$\mathcal{L}_{cov} = \max(0, \rho - C_t(\hat{R}_t)) + \lambda \frac{|\hat{R}_t|}{K_{max}}$$

这个 loss 更贴近最终目标。

但它涉及 Top-K，不完全可导。可以用两种方式：

1. 作为 evaluation objective；
2. 用 soft top-k approximation；
3. 训练时仍用 label loss，后期加这个作为 fine-tuning objective。

MVP 不必做这个。

11. 我建议最终采用的 loss 组合

Training

$$\mathcal{L}_{train} = \lambda_1 \mathcal{L}_K^{ord} + \lambda_2 \mathcal{L}_{reuse}^{BCE} + \lambda_3 \mathcal{L}_{cat}^{KL} + \lambda_4 \mathbb{E}[K_t]$$

其中：

- $\mathcal{L}_K^{ord}$ ：预测 block budget；
- $\mathcal{L}_{reuse}^{BCE}$ ：预测是否复用；
- $\mathcal{L}_{cat}^{KL}$ ：预测 block 类别分布；
- $\mathbb{E}[K_t]$ ：成本正则。

Test-time adaptation

不训练主模型，只更新 calibration：

$$\theta_{online} = \{b_K, b_R, b_C\}$$

使用 proxy objective：

$$\mathcal{L}_{test} = \alpha \mathcal{L}_{under} + \beta \mathcal{L}_{over} + \gamma \mathcal{L}_{reuse-fail} + \delta \mathcal{L}_{cat-align}$$

其中：

$$\mathcal{L}_{under} = \max(0, P_t^{tail} - \tau_{tail}) + \max(0, \tau_{\Delta} - \Delta_t)$$

$$\mathcal{L}_{over} = \max(0, \tau_u - u_t) + \lambda_K \frac{K_t}{K_{max}}$$

$$\mathcal{L}_{reuse-fail} = 1[reuse](H_t^{sparse} + P_t^{tail} + d_t)$$

$$\mathcal{L}_{cat-align} = \text{KL}(\hat{A}_t^{cat} \| p_t^{cat})$$

12. 关键判断：test-time 不要说“监督学习”，要说“self-calibration”

测试时没有 oracle supervision，所以不要写成 supervised online learning。更准确的表述是：

During inference, the policy is adapted through self-calibration signals derived from sparse attention statistics and retrieval stability.

也就是说，测试时用的是：

- sparse attention feedback;
- score boundary uncertainty;
- retrieval set temporal stability;
- query drift;
- category utilization;
- generation confidence。

它们不是 ground-truth label，而是 **self-supervised proxy signals**。

13. 最终技术路线建议

我建议你这样定：

训练阶段

使用 full attention oracle 构造：

1. budget label:

$$K_t^* = \min K \quad s.t. \quad \sum_{b \in TopK} A_{t,b}^{full} \geq \rho$$

2. reuse label:

$$y_t^{reuse} = 1 \quad \text{if} \quad \sum_{b \in O_{t-1}} A_{t,b}^{full} \geq \rho_{reuse}$$

3. category label:

$$y_t^{cat}(j) = \sum_{b:c(b)=j} A_{t,b}^{full}$$

训练 loss:

$$\mathcal{L}_{train} = \lambda_1 \mathcal{L}_K^{ord} + \lambda_2 \mathcal{L}_{reuse}^{BCE} + \lambda_3 \mathcal{L}_{cat}^{KL} + \lambda_4 \mathbb{E}[K_t]$$

测试阶段

使用 sparse attention 可观测信号：

1. tail pressure;
2. utilization;
3. score boundary margin;
4. sparse attention entropy;
5. query drift;
6. retrieved set overlap;
7. category attention mass。

在线更新：

$$b_K, b_R, b_C$$

不更新 LLM, 不更新完整 policy MLP。

测试 proxy loss:

$$\mathcal{L}_{test} = \alpha \mathcal{L}_{under} + \beta \mathcal{L}_{over} + \gamma \mathcal{L}_{reuse-fail} + \delta \mathcal{L}_{cat-align}$$

我认为这个设计比较稳：**训练时有强 oracle 监督；测试时有低成本 self-calibration；整体仍然是 block-level retrieve，不会偏到 token/window/CoT 复杂范式。**